Installing Foreman/Katello on RHEL 9

A homelab troubleshooting journal: every error we hit, why it happened, and how it was fixed

This document walks through a real, end-to-end install of Foreman (the open-source project behind Red Hat Satellite) with the Katello plugin on a RHEL 9 VM in a VMware ESXi homelab. It's written in plain language so that if you hit any of these errors again — on this box or a fresh one — you can recognize the symptom and jump straight to the fix, without re-diagnosing everything from scratch.

Each section below follows the same pattern: **What happened**, **Why it happened**, and **How we fixed it**, plus a short note on how to avoid it next time.

What's in this guide

1. Shell variables not carrying over between commands

What happened

We set two variables (FOREMAN_VERSION and OS_MAJOR_VERSION) and then ran the install commands, but the download URLs came back broken — things like **releases//el/x86_64** with gaps where the version numbers should have been.

Why it happened

The variables were set in one terminal session, but the actual install commands were run in a different session (for example, after closing and reopening the terminal, or in a separate tab). Once a terminal session ends, anything you "exported" in it disappears.

How we fixed it

We re-typed both variables and immediately checked them with echo, in the exact same terminal window, before running anything else:

```
export FOREMAN_VERSION=3.13
export OS_MAJOR_VERSION=9

echo $FOREMAN_VERSION
echo $OS_MAJOR_VERSION
```

Once both echoed back the correct numbers, we ran the install commands right after, in that same window, without switching tabs or reopening the terminal in between.

How to avoid this next time: Always set a variable and use it in the same terminal session, back to back. If you must switch windows, re-export the variable again before continuing.

2. Missing EPEL repository on RHEL

What happened

Running **sudo dnf install -y epel-release** failed with "No match for argument: epel-release" and "Unable to find a match".

Why it happened

Unlike CentOS or Rocky Linux, plain RHEL does not include an installable package simply called "epel-release" in its default repositories. EPEL has to be added by pointing dnf directly at the actual file from the Fedora project's website.

How we fixed it

We installed EPEL using its direct download link instead of the short package name:

```
sudo dnf install -y https://dl.fedoraproject.org/pub/epel/epel-release-latest-9.noarch.rpm
```

How to avoid this next time: On real RHEL (not CentOS/Rocky), always add EPEL using the full URL from the Fedora project, not just the short name.

3. Ruby "module" version conflict (first appearance)

What happened

The installer failed partway through with an error saying a package needed **rubygem(rdoc) < 6.4**, but the only rdoc versions available were "filtered out by modular filtering".

Why it happened

RHEL lets you choose between different bundled versions of certain software (called "modules") — for example, Ruby 3.0 vs Ruby 3.1. We had switched to the newer Ruby 3.1 module, but some required packages were still tied to the older Ruby 3.0 module. Once you tell RHEL to use one version, it refuses to touch packages tied to the version you're not using — that's what "filtered out by modular filtering" means.

How we fixed it

We checked which Ruby version was active:

```
sudo dnf module list ruby --enabled
```

Since packages needed the older stream, we used switch-to (not just enable) to fully move the system back, including already-installed packages:

```
sudo dnf module switch-to ruby:3.0 -y
sudo dnf distro-sync -y
```

How to avoid this next time: If you see the words "filtered out by modular filtering", it almost always means two different module versions (like Ruby 3.0 vs 3.1) are in conflict. Check with "dnf module list <name> --enabled" and use "switch-to" (not just "enable") to fully move an already-partly-installed system.

4. Pressing Ctrl+C during the install left things half-done

What happened

The installer was progressing nicely (over 1000 out of about 1360 steps done), but it was interrupted midway to check something else, using Ctrl+C. On the next run, new errors appeared that hadn't been there before.

Why it happened

The Foreman installer configures hundreds of interconnected settings and packages in one long run. Stopping it partway through can leave some pieces configured and others not, which can look like new problems on the next attempt even though nothing else changed.

How we fixed it

The installer is designed to be safely run again and again (this is called being "idempotent") — so the fix was simply to let it run all the way to the end without interrupting it, and to use a second terminal window to check on progress instead of stopping the main one.

```
tail -f /var/log/foreman-installer/katello.log
```

How to avoid this next time: Never press Ctrl+C on a long-running installer unless it's truly stuck. If you want to check progress, open a second terminal/SSH window and read the log file there instead.

5. Ruby 3.0 was completely missing from the system

What happened

After the Ctrl+C interruption, we tried to switch back to Ruby 3.0 again, but this time got a hard error: "missing groups or modules: ruby:3.0" — meaning the option to even select that version was gone.

Why it happened

RHEL periodically retires older bundled software versions from its official update servers as time passes (this is normal and expected over a system's lifetime). By the time we tried to switch back, Ruby 3.0 had been fully removed from what RHEL 9 currently offers — only 3.1, 3.3, and 4.0 remained as options. You cannot install a version that the update server no longer provides at all.

How we fixed it

We confirmed this with:

```
sudo dnf module list ruby
```

Since Ruby 3.0 was gone for good, going backward was no longer possible. This meant the actual fix had to be a different one — explained in the next section.

How to avoid this next time: If a version you need has vanished entirely from "dnf module list", don't keep trying to switch back to it — it's gone from the update server, not just disabled. The real fix is to move forward to software that supports the newer version already on your system.

6. The real wall: Katello wasn't yet built for the newer Ruby version

What happened

Even after trying a newer version of Foreman and Katello, the install still failed with the same kind of message: certain Katello packages explicitly "require ruby < 3.1", and certain other packages needed libraries that only exist for the older Ruby.

Why it happened

This wasn't a mistake we made — it was a genuine, current gap between two projects moving at different speeds. RHEL 9 had already moved its default Ruby forward, but the version of Katello available at the time still had some pieces compiled specifically for the older Ruby. Neither project had "caught up" with the other yet on this particular RHEL update.

How we fixed it

We searched Foreman's own documentation and community forums to confirm this was a known, structural limitation rather than something fixable by tweaking settings, and confirmed there was a different, better

installation method available (see the next section).

How to avoid this next time: If an error explicitly says a package "requires ruby < X" (or similarly names an exact version ceiling) and no version switch fixes it, stop trying to force it — check the project's official docs or community forum for a currently-recommended install method instead of continuing to patch around it.

7. Switching to the container-based installer (foremanctl)

What happened

Rather than keep fighting the system Ruby version, we switched to Foreman's newer container-based install method, called **foremanctl**.

Why it happened

This method runs Foreman and Katello inside self-contained "containers" (think of them as sealed boxes that carry their own copy of Ruby and all their dependencies pre-packaged inside). Because each container brings its own Ruby version, it doesn't matter what version RHEL itself currently has — there's no more tug-of-war between the OS and the application.

How we fixed it

We first removed the old, broken install attempt:

```
sudo dnf remove -y foreman-installer-katello katello foreman-proxy foreman-postgresql \
foreman-redis foreman-dynflow-sidekiq
sudo rm -f /etc/yum.repos.d/foreman*.repo /etc/yum.repos.d/katello*.repo
/etc/yum.repos.d/puppet*.repo
sudo dnf clean all
```

Then enabled the container-based installer's repository and installed it:

```
sudo dnf copr enable @theforeman/foremanctl rhel-9-x86_64 -y
sudo dnf upgrade -y
sudo dnf install -y foremanctl
```

Then started the deployment:

```
sudo foremanctl deploy
```

How to avoid this next time: If a project offers both a traditional (package-based) installer and a newer container-based one, and you're hitting deep OS-version conflicts, the container-based option is often the more reliable choice — it sidesteps host operating system version mismatches almost entirely. Note: this method was still labeled a 'Technology Preview' at the time, meaning it's newer and not yet the fully polished official path, but it worked well for homelab purposes.

8. Corrupted PostgreSQL (database) data folder

What happened

The database container failed to start, with the error: "'/var/lib/pgsql/data' is not a valid data directory" and "File PG_VERSION is missing".

Why it happened

An earlier, failed install attempt had already partly written files into that same folder. When the new database container tried to start fresh, it found a folder that was neither empty (which it needs, to set up from scratch) nor a complete, valid database (which it needs, to just start up normally). It was stuck in between.

How we fixed it

We stopped the service, completely emptied that folder, and let it reinitialize from a clean state:

```
sudo systemctl stop postgresql.service
sudo rm -rf /var/lib/pgsql/data/*
sudo restorecon -Rv /var/lib/pgsql/data/
sudo systemctl start postgresql.service
sudo systemctl status postgresql.service
```

Afterward, the status showed the database "accepting connections", confirming it had reinitialized correctly.

How to avoid this next time: If you ever re-run an install after a previous attempt failed partway through, and a database container specifically complains its data folder is "invalid" or missing a version file, it's usually safe (in a fresh homelab setup with no real data yet) to clear that folder completely and let it start over.

9. A leftover Redis program fighting with the new container

What happened

The very last step — "Wait for Foreman service to be accessible" — kept failing after many retries, with an error buried in the response about Redis (a small support service Foreman depends on) being unable to save its data to disk.

Why it happened

Earlier in the process, a plain, ordinary copy of Redis had been installed directly onto the operating system (as a side effect of an earlier troubleshooting step). This older copy was still running under its own name and grabbed control before the new, properly-configured container version could start. The old copy didn't have the right folder permissions set up, so it couldn't save its data — and that failure was what was blocking Foreman's health check.

How we fixed it

We removed the old, plain copy of Redis entirely, and let the container-managed version take over cleanly:

```
sudo systemctl stop redis.service
sudo systemctl disable redis.service
sudo dnf remove -y redis
sudo systemctl daemon-reload
sudo systemctl reset-failed
sudo systemctl start redis.service
```

We then confirmed the container version was running properly with clean logs, showing "Ready to accept connections" with no errors.

How to avoid this next time: If a background service seems to be running (systemd says "active") but doesn't show up in your container list at all, it's likely a plain, non-container copy of that same program installed separately and quietly conflicting with the container version. Check with "rpm -qa | grep <name>" to see if a native package snuck in.

10. Missing memory setting for Redis (preventive fix)

What happened

Once Redis's logs were visible properly, they showed a warning: "Memory overcommit must be enabled", flagging that background saves could fail under certain memory conditions — the exact class of problem we'd just fixed.

Why it happened

Redis periodically saves a snapshot of its data to disk in the background. To do this safely without risking the whole system running out of memory, Linux has a setting that needs to explicitly allow this behavior. By default, that setting wasn't turned on.

How we fixed it

We turned the setting on immediately, so it takes effect without needing a restart:

```
echo 'vm.overcommit_memory = 1' | sudo tee -a /etc/sysctl.conf
sudo sysctl vm.overcommit_memory=1
```

How to avoid this next time: When you see a warning in a service's logs (even if the service seems to be working right now), fix it immediately rather than waiting for it to cause a real failure later — warnings like this are often an early heads-up about the exact same kind of problem you may have just spent an hour debugging.

11. Success! Where to find your login details

What happened

After all the fixes above, running the deploy one final time completed with zero failures.

Why it happened

Every earlier blocker (Ruby version mismatch, corrupted database folder, conflicting Redis copy, missing memory setting) had been resolved, so the installer could finally get all the way through every step without hitting a wall.

How we fixed it

The final output included a message like this, which contains your actual login details — save this somewhere safe, since it won't be shown again automatically:

```
Foreman is running at https://satellite.homelab.local
Admin credentials: admin:YOUR-GENERATED-PASSWORD
```

The play recap line at the very end (showing something like "failed=0") is the clearest confirmation that everything finished cleanly.

How to avoid this next time: As soon as you see your admin password printed like this, copy it into a password manager right away. Then log in once and change it to something you'll remember, since the auto-generated one is hard to retype.

12. Opening the site from your Windows computer

What happened

Typing the address (something like https://satellite.homelab.local) into a browser on the Windows computer either couldn't find the site, or timed out.

Why it happened

The web address ends in ".local", which is not a real, publicly registered domain name — it only means something if you specifically tell your computer what IP address it points to. Windows needs a small manual entry (in its "hosts" file) to know where to send that request.

How we fixed it

We found the server's actual network address on the RHEL machine:

```
ip addr show | grep "inet " | grep -v 127.0.0.1
```

On the Windows computer, we opened Notepad as Administrator, then used File > Open and typed the exact path directly into the address bar (this avoids Notepad's file filter hiding the file, since it has no ".txt" ending):

```
C:\Windows\System32\drivers\etc\hosts
```

Then added one line at the bottom of that file (using the real IP address found earlier), and saved it:

```
192.168.50.3 satellite.homelab.local
```

After that, typing https://satellite.homelab.local into the browser successfully found the login page (after clicking through a one-time "not private" warning, which is expected and safe for a homelab server using a self-made security certificate).

How to avoid this next time: Whenever you use a made-up ".local" or custom address for a homelab server, remember that every computer that needs to reach it — not just the server itself — needs that same manual hosts file entry pointing to the right IP address.

13. General lessons for next time

- When you set a shell variable, use it right away in the same terminal window — don't switch windows or reopen the terminal in between, or it will silently disappear.

- On real RHEL (not CentOS/Rocky), remember EPEL needs its full download link, not just a short package name.
- The phrase "filtered out by modular filtering" almost always means two software version streams (like two different Ruby versions) are in conflict. Check with "dnf module list" and use "switch-to" to move cleanly between them.
- Never interrupt a long-running installer with Ctrl+C unless it's genuinely stuck. Use a second terminal window to check progress instead.
- If a required older software version has disappeared completely (not just "disabled"), you can't go backward — look for a newer method or version that matches what's currently available instead.
- If an error names an exact version ceiling (like "requires ruby less than 3.1") and switching versions doesn't fix it, that's usually a real, current limitation of the software itself — check the official docs or forums for the currently-recommended approach rather than continuing to patch around it.
- A container-based installer (one that bundles its own dependencies) is often more reliable than a traditional package-based one when you're fighting operating-system version mismatches.
- If a fresh install fails partway through and you retry it, and a database service complains its data folder is invalid or incomplete, it is usually safe (with no real data yet) to clear that folder and let it start over from scratch.
- If a background service is reported as "running" by systemd but doesn't appear in your container list, check whether a plain, non-container copy of that same program was installed separately and is quietly taking priority.
- Pay attention to warnings in service logs even when things seem to be working — they often predict the exact failure you'll hit later.
- Save any auto-generated admin password immediately into a password manager — it's usually shown only once.
- Any custom ".local" address for a homelab server needs a manual hosts file entry on every computer that will connect to it, pointing to the server's real network address.